Good question. The term **Workflow Selector** can be confusing because in the newer agentic architecture it is **not the same as a traditional workflow engine**.

The purpose of a **Workflow Selector** is to decide **which business process or "mode of operation" applies**, before the AI agent starts reasoning deeply.

Think of it as a **routing layer**, not the executor.

Why do we need a Workflow Selector?

A contact center has many possible business processes:

```
Customer Support
    ├── Technical Issue
    ├── Billing Issue
    ├── Refund
    ├── Cancellation
    ├── Upgrade
    └── Complaint
```

The system needs to know:

> "Which business context am I operating in?"

before applying the right tools, policies, and guardrails.

Example

Customer says:

> "My internet stopped working yesterday, and I was charged for a service I never used."

The Intent Detector extracts:

```
{
  "intents": [
    "technical_issue",
    "billing_dispute"
```

```
  ]
}
```

Now the Workflow Selector decides:

```
Primary workflow:
Technical Support

Secondary workflow:
Billing Investigation

Priority:
Resolve connectivity first
```

Then the agent operates within that context.

What happens without Workflow Selector?

The LLM sees:

```
Customer message
+
100 available tools
+
50 policies
+
20 workflows
```

and has to figure everything out every turn.

Problems:

- Higher hallucination risk

- More tokens
- Harder governance
- Harder debugging
- Inconsistent behavior

Traditional vs Agentic approach

Traditional contact center

The flow is:

```
User Transcript

↓

Intent Classifier

↓

Workflow Selector

↓

Fixed Workflow

↓

Actions
```

Example:

```
Refund Request
```

```
↓

Refund Workflow

↓

Check eligibility

↓

Process refund
```

Very predictable.

Agentic contact center

The flow is:

```
User Transcript

↓

Intent Understanding

↓

Context Selection

↓

AI Agent Planner

↓
```

```
Dynamic Plan

↓

Tools + RAG + Workflows
```

The workflow selector becomes more like:

> "What domain/process should guide the reasoning?"

not:

> "Follow these exact 10 steps."

A better internal architecture diagram

I would actually rename it to avoid confusion:

```
User Transcript

        ↓

Intent + Entity Understanding

        ↓

Business Context Router
        |
        |
        ├── Technical Support Context
        |
        ├── Billing Context
        |
```

```
        ├── Sales Context
        │
        └── Complaint Context

        ↓

Conversation State Update

        ↓

AI Agent Planner

        ↓

Dynamic Execution Plan

        ↓

 ┌──────────────┬──────────────┬──────────────┐
 │              │              │
Tools          RAG        Human Escalation
 │              │              │
CRM/API      Knowledge     Agent Queue

        ↓

Response Generation

        ↓

TTS
```

So where does Workflow Engine fit now?

There are actually **three different concepts**:

Component	Purpose	Example
Business Context Router (Workflow Selector)	Decide the domain/process	"This is a refund case"
AI Agent Planner	Decide steps dynamically	"Get invoice → check policy → offer refund"
Workflow Executor	Run deterministic processes	"Refund approval flow"

For your contact center demo, I would avoid the term **Workflow Selector** because it implies old-school RPA. I would use:

"Business Context Router"

or

"Intent & Process Router"

because it better reflects the role in a modern AI agent architecture.

Fri, Jul 31 at 5:55 AM

tell me such complex queries like "My internet stopped working yesterday, and I was charged for a service I never

used." which can be used as test cases. Just give the questions.. nothig else

1. "My internet has been slow for the past week, I was billed extra this month, and I am thinking of cancelling because this keeps happening."
2. "I was charged twice for my last payment, my service is still not active, and I need this fixed before tomorrow."
3. "I want to upgrade my plan because my current one is too slow, but I also noticed an unexpected charge on my latest bill."
4. "My delivery is delayed, I was promised it would arrive yesterday, and I want to know if I can get a refund."
5. "I cancelled my subscription last month, but I was charged again today and I want my money back."
6. "My phone was stolen, I need to block the SIM, transfer my number to a new device, and check if someone used my data."
7. "I am moving to a new house next week, I need my service transferred, but I also want to know if my current plan is still the best option."
8. "My payment failed yesterday, but the money has already been deducted from my bank account. Can you check what happened?"
9. "I have been having connection issues since the technician visited, and I was also charged for the service visit even though the problem was not fixed."
10. "I want to cancel my account, but before I do that, tell me what charges I will have to pay and whether there is a better plan available."
11. "My bill is much higher than usual. I travelled last month, but I don't understand these roaming charges. Can you explain and help me reduce the bill?"
12. "I ordered a new device, the payment went through, but I received the wrong model. I want an exchange and a refund for the price difference."

13. "I have already contacted support twice about my issue, nobody resolved it, and I want this escalated to someone who can actually help."
14. "My service stopped working while I was travelling, I need to know if it is a network problem or something wrong with my account."
15. "I want to upgrade my internet speed, but first check whether my area supports the higher plan and whether I qualify for any discounts."
16. "I was promised a promotional discount when I signed up, but my first bill does not include it. Can you check my agreement and fix it?"
17. "My account was charged after I returned the equipment, and I want to know why there is still an outstanding balance."
18. "I need to add a family member to my account, but I also want to change the payment method and understand the impact on my bill."
19. "The app shows my payment as pending for three days, my service is restricted, and I need to know when it will be restored."
20. "I am unhappy with the service quality, I want compensation for the outage, and if that is not possible I want to cancel."
21. "My WiFi works on some devices but not others, I already restarted everything, and I need help troubleshooting this."
22. "I received an email saying my plan is changing next month. I want to know why, what the new charges are, and whether I can opt out."
23. "I need a copy of my previous invoices for tax purposes, but I also noticed one invoice has incorrect charges."
24. "My account was suspended even though I paid the bill. Please check my payment, restore my service, and tell me what caused the suspension."
25. "I want to switch providers, but before I leave, tell me if you can offer a better plan based on my usage history."

Today 8:53 AM

> Can you provide a complete detailed summary of the architecture of the command center that can be used to create workflow and timelines.

Yes. Based on the architecture we converged on, I would treat the **AI Contact Center / Command Center** as a small platform rather than a single voice-bot application.

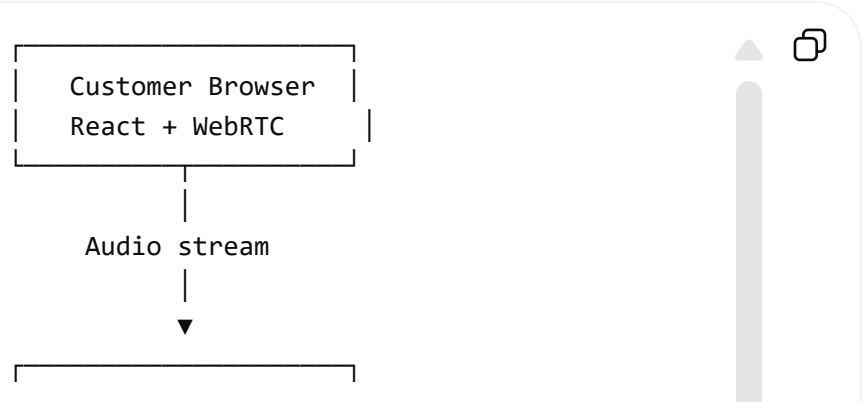
The key architectural principle is:

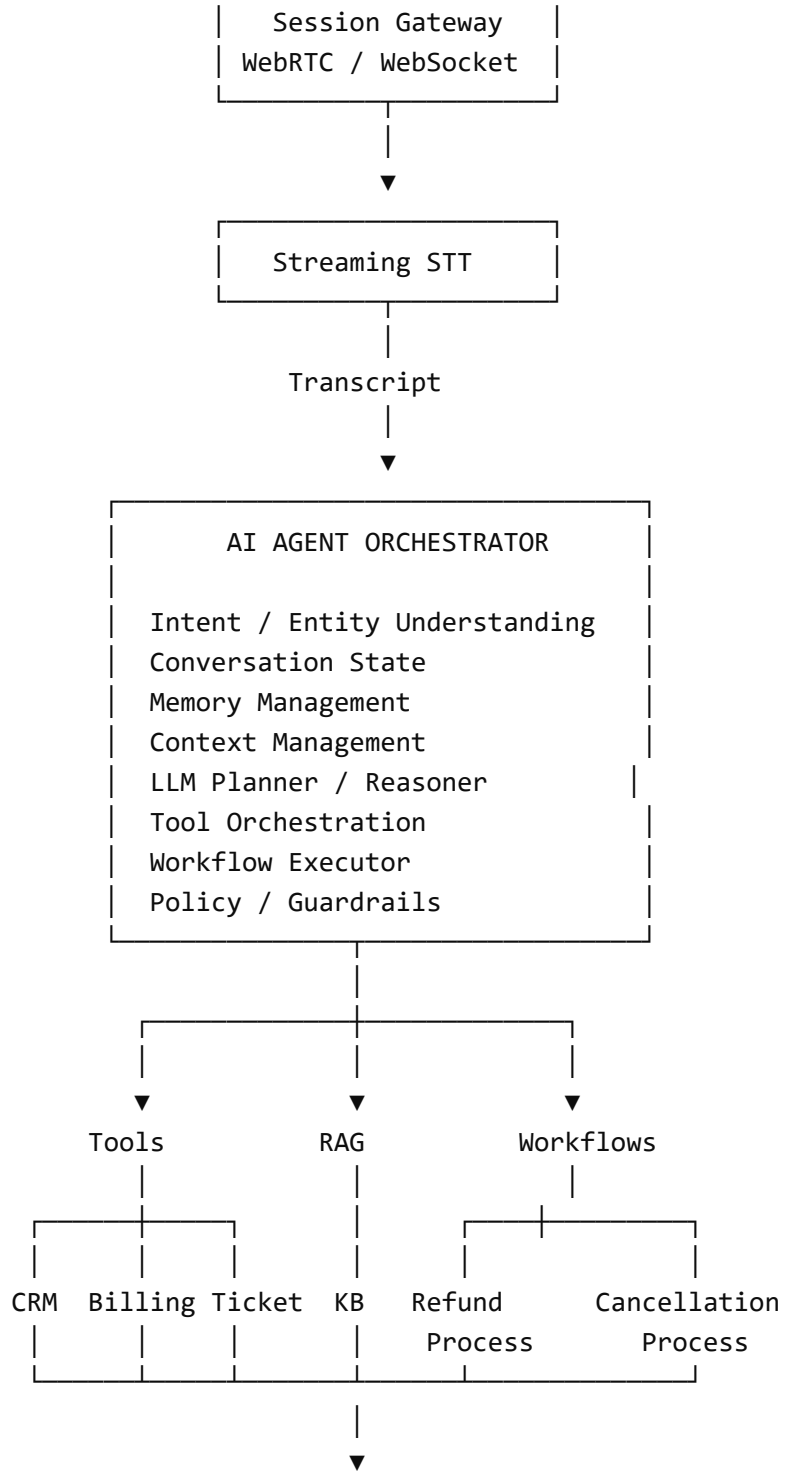
> **The LLM is the dynamic reasoning/planning layer; deterministic workflows are controlled execution paths inside the agent; tools and enterprise systems perform real actions; memory maintains continuity; RAG supplies knowledge; policy/guardrails constrain the agent.**

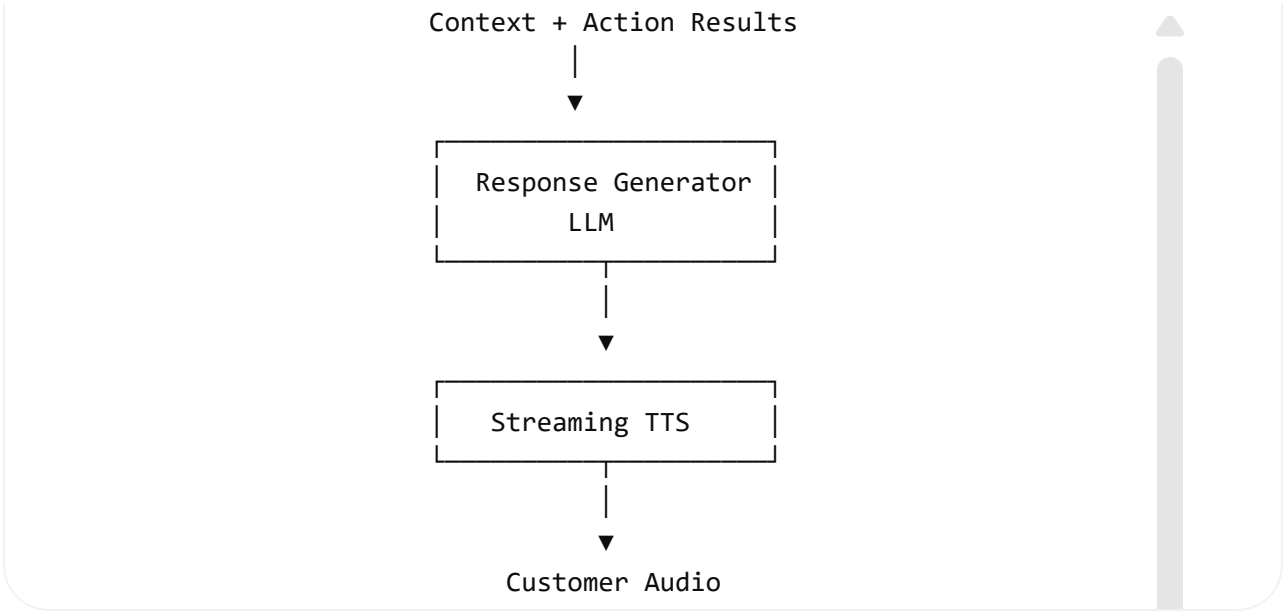
Below is a version that can directly be converted into a development backlog, work breakdown structure, and timeline.

1. Target system

The demo should support this end-to-end interaction:




Session Gateway
WebRTC / WebSocket
Streaming STT
Transcript
AI AGENT ORCHESTRATOR
Intent / Entity Understanding
Conversation State
Memory Management
Context Management
LLM Planner / Reasoner
Tool Orchestration
Workflow Executor
Policy / Guardrails
Tools
RAG
Workflows
CRM
Billing
Ticket
KB
Refund Process
Cancellation Process



There is also an **out-of-band observability and Command Center UI** connected to essentially every layer.

---

2. The Command Center itself

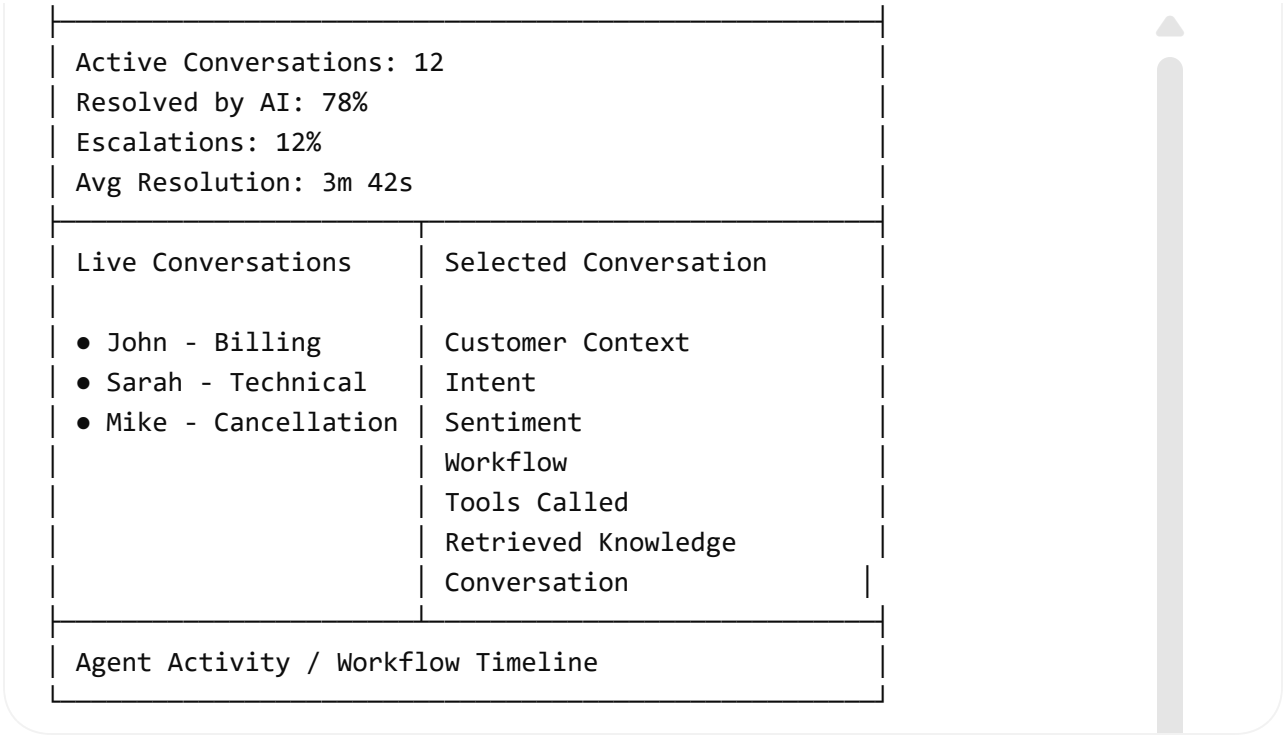
I would separate the **customer conversation UI** from the **Command Center UI**.

The customer sees:

```
Start Conversation
     ↓
Voice conversation
```

The operator/demo user sees:

```
┌──────────────────────────────────────────────────────────┐
│                    AI CONTACT CENTER                     │
```



This is important because the **Command Center is where you demonstrate the complexity**.

The audience should be able to see:

> Customer said X → Agent understood Y → retrieved Z → called three systems → applied policy → completed transaction.

---

3. Core architecture components

A. Customer Experience Layer

Browser application

Responsibilities:

- microphone access
- WebRTC connection
- audio playback
- connection status
- transcript display
- call controls
- interruption/barging
- conversation state display where appropriate

Technology:

- React
- TypeScript
- WebRTC
- WebSocket for event/control messages

Output

The customer experiences a normal voice conversation.

---

4. Session Gateway

The Session Gateway is deliberately separate from the AI agent.

Responsibilities:

- create session
- maintain WebRTC/WebSocket connection
- authenticate session

- route audio
- correlate audio with conversation ID
- publish events
- handle reconnects
- terminate sessions

Core entity:

```
ConversationSession
-------------------
session_id
customer_id
start_time
channel
status
language
agent_id
```

Everything downstream should operate using a common `session_id`.

---

5. Speech layer

Streaming STT

Responsibilities:

- streaming transcription
- partial transcripts
- final transcripts
- timestamps
- speaker information

- optional language detection

Important event:

```
{
  "event": "transcript.final",
  "session_id": "S123",
  "text": "My internet stopped working yesterday",
  "timestamp": "..."
}
```

The AI system should consume **events**, not tightly couple itself to the speech provider.

That gives you the ability to replace Azure Speech with Deepgram, etc.

---

6. AI Agent Orchestrator

This is the most important subsystem.

Do not implement this as one giant prompt.

Break it into explicit capabilities.

```
AI Agent Orchestrator
│
├── Intent & Entity Understanding
├── Conversation State
├── Memory Manager
├── Context Manager
├── Planner / Reasoner
├── Tool Orchestrator
```

```
├── Workflow Executor
├── RAG Manager
├── Policy / Guardrails
└── Response Generator
```

---

7. Intent and entity understanding

The system extracts:

```
{
  "intent": [
    "technical_issue",
    "billing_issue"
  ],
  "entities": {
    "service": "internet",
    "time": "yesterday"
  },
  "sentiment": "frustrated",
  "urgency": "medium"
}
```

Important distinction:

Intent detection is not the final decision maker.

It provides context to the planner.

For example:

```
Technical issue
```

```
+
Billing issue
+
Cancellation threat
```

The planner can determine that several capabilities are required.

---

8. Memory Management

This should be a first-class service, not merely conversation history.

Short-term conversational memory

```
Customer already restarted router.
Customer already gave account number.
Customer was asked about outage.
```

Working memory / task state

```
Customer verified = true
Diagnostics complete = true
Engineer booking = pending
```

Long-term customer memory

```
Previous issue
Previous tickets
Plan
Preferences
Prior interactions
```

Recommended separation:

```
Redis
    ↓
Current session / working state

PostgreSQL
    ↓
Persistent conversation data

Customer/CRM
    ↓
Enterprise customer history
```

Do not put everything into the LLM prompt.

Build a **Context Manager** that selects what the LLM actually needs.

---

9. Context Manager

This becomes very important as conversations become complex.

Suppose the customer has:

- 200 historical messages
- 5 tickets
- 12 invoices
- 3 products
- 10 previous calls

The LLM should not receive all of it on every turn.

The context manager assembles:

```
Current utterance
+
Relevant conversation history
+
Current task state
+
Relevant customer profile
+
Tool results
+
Relevant knowledge
+
Applicable policies
```

That becomes the reasoning context.

---

10. Planner / Reasoner

This is where your architecture becomes agentic.

The planner receives:

```
Customer request
Current state
Available capabilities
Policies
Previous actions
```

and determines the next action.

Example:

```
User:
"My internet stopped yesterday, the bill has an extra charge,
and if this isn't fixed I want to cancel."

Planner:

1. Retrieve customer
2. Check service status
3. Retrieve current invoice
4. Retrieve relevant billing policy
5. Explain issue
6. Determine whether technical resolution is required
7. If unresolved, offer engineer appointment
8. If customer still requests cancellation,
   invoke cancellation workflow
```

This plan can change based on tool results.

That is the critical difference from a fixed workflow.

---

11. Tool Orchestrator

Every enterprise action becomes a typed tool.

Example:

```
get_customer()
get_account()
get_invoice()
get_order()
check_outage()
```

```
run_diagnostics()
create_ticket()
schedule_engineer()
cancel_service()
issue_refund()
send_sms()
send_email()
```

Each tool should have:

```
name
description
input schema
output schema
authorization rules
timeout
retry policy
audit information
```

The LLM should **never have unrestricted access to APIs**.

It selects a tool.

The Tool Orchestrator validates and executes it.

---

12. Workflow Executor

This is where deterministic workflows still matter.

For example:

Refund

```
Verify identity
       ↓
Check transaction
       ↓
Check refund eligibility
       ↓
Check amount threshold
       ↓
Approval if necessary
       ↓
Issue refund
       ↓
Send confirmation
```

The AI planner can decide:

> "A refund workflow needs to be executed."

But once inside that workflow, critical steps can remain deterministic.

So the relationship is:

```
LLM Planner
     ↓
"Run refund workflow"
     ↓
Workflow Executor
     ↓
Deterministic business process
```

This is much safer than asking an LLM to independently decide every financial operation.

---

13. RAG subsystem

RAG should be a **knowledge capability**, not the universal solution.

Use it for:

```
Refund policy
Warranty
Product documentation
Service policy
Cancellation policy
Terms & conditions
Troubleshooting procedures
```

RAG pipeline:

```
Documents
   ↓
Chunking
   ↓
Embeddings
   ↓
Vector/Search Index
   ↓
Retrieval
   ↓
Reranking
   ↓
Relevant passages
   ↓
LLM
```

For the demo, support:

- document upload

- indexing
- semantic search
- metadata filters
- citations
- source display

The Command Center should show which documents were retrieved.

---

14. Policy and Guardrails

This is another first-class subsystem.

It controls:

```
What can the agent say?
What can it do?
What requires confirmation?
What requires human approval?
What must be escalated?
```

Examples:

```
Refund > ₹10,000
        ↓
Manager approval

Customer requests cancellation
        ↓
Retention policy applies

Fraud suspicion
        ↓
```

```
No autonomous action

Identity not verified
        ↓
No sensitive account operations
```

The planner proposes.

The policy engine authorizes.

The tool layer executes.

That separation is important.

---

15. Response Generator

Once actions are complete:

```
User request
+
Conversation context
+
Tool results
+
Knowledge
+
Policy constraints
+
Task state
```

go to the response LLM.

Example:

```
"I found the additional ₹850 charge. It came from roaming
usage on 12 August. Your plan does not include that roaming
package. I can see that you have already been experiencing
connectivity issues, so I can arrange a technician visit
for tomorrow at 10 AM."
```

Then TTS.

---

16. Observability

Every interaction should produce an event stream.

Example:

```
08:32:10  speech.final
08:32:11  intent.detected
08:32:11  customer.lookup.started
08:32:12  customer.lookup.completed
08:32:12  invoice.lookup.started
08:32:13  invoice.lookup.completed
08:32:13  rag.search.started
08:32:14  rag.search.completed
08:32:14  planner.decision
08:32:15  response.generated
08:32:15  tts.started
```

This event stream powers the Command Center.

---

17. Command Center UI modules

I would build these screens.

1. Live Operations Dashboard

Metrics:

```
Active conversations
AI containment
Escalation rate
Average handling time
Average response latency
Tool failure rate
Customer sentiment
```

2. Conversation Monitor

```
Transcript
Customer
Intent
Sentiment
Memory
Current task
Planner decision
Tools
RAG sources
Policy decisions
```

3. Workflow / Agent Timeline

This is particularly valuable.

Example:

```
Customer speaks
```

```
      ↓
Intent detected
      ↓
CRM lookup
      ↓
Invoice retrieved
      ↓
Policy retrieved
      ↓
Planner updated
      ↓
Refund workflow started
      ↓
Refund completed
      ↓
SMS sent
```

4. Knowledge Explorer

Show:

```
Question
        ↓
Retrieved documents
        ↓
Relevant passages
        ↓
Answer
```

5. Tool/Integration Monitor

Show:

```
CRM          ✓ healthy
Billing      ✓ healthy
```

```
Ticketing     ✓ healthy
Scheduling    ✓ healthy
```

---

18. Suggested data model

At minimum:

```
Customer
Account
Conversation
Message
ConversationState
Memory
Intent
Workflow
WorkflowExecution
Tool
ToolExecution
KnowledgeDocument
KnowledgeRetrieval
Policy
PolicyDecision
AgentAction
Escalation
CallSummary
```

The most important relationship is:

```
Conversation
    │
    ├── Messages
    ├── State
    ├── Memory
```

```
├── Intents
├── Planner decisions
├── Tool executions
├── RAG retrievals
├── Policy decisions
├── Workflow executions
└── Summary
```

This makes the entire call replayable.

---

19. Recommended technology stack

For a practical demo:

Layer	Technology
Frontend	React + TypeScript
Voice transport	WebRTC
Session/API	FastAPI
Event transport	WebSocket
STT	Azure Speech / Deepgram
LLM	GPT / Claude / Gemini
Agent orchestration	LangGraph or custom stateful orchestrator
Redis	session + working state

Layer	Technology
PostgreSQL	persistent data
RAG	Azure AI Search / pgvector
TTS	Azure Speech / ElevenLabs
Mock enterprise systems	FastAPI
Observability	OpenTelemetry + Langfuse
Containerization	Docker
Deployment	Kubernetes or simple container hosting for demo

I would avoid over-engineering Kubernetes initially. Docker Compose is enough for the first development environment.

---

20. Development workstreams

This is how I would split the project for parallel development.

Workstream	Scope
W1	Frontend + WebRTC
W2	Session Gateway + realtime events
W3	STT/TTS

Workstream	Scope
W4	AI Agent Orchestrator
W5	Memory + State
W6	Tools + Mock enterprise APIs
W7	RAG
W8	Workflow Executor
W9	Policy/Guardrails
W10	Command Center UI
W11	Observability
W12	Demo scenarios + integration

---

21. Proposed delivery timeline

For a small team of roughly 4–6 engineers, I would target **6–8 weeks** for a polished demo.

Week 1 — Foundation

Deliver:

```
React application
WebRTC connection
Backend skeleton
```

```
Session IDs
Event schema
Docker environment
Basic STT/TTS
```

End-of-week demo:

> Customer speaks → transcript appears → AI speaks back.

Week 2 — Basic Agent

Build:

```
Intent detection
Conversation state
Basic LLM response
Session memory
Streaming response
```

End-of-week demo:

> Multi-turn conversation with context.

Week 3 — Tools and Enterprise Systems

Build mock:

```
CRM
Billing
Ticketing
```

```
Scheduling
```

Add:

```
Tool schemas
Tool router
Tool execution
Tool result handling
```

End-of-week demo:

> Customer asks a transactional question and the agent actually changes backend state.

Week 4 — Agentic Planning

Build:

```
Planner
Dynamic tool selection
Multi-step tool calls
Context manager
Planning loop
Error handling
```

This is the week where your demo moves from:

voice chatbot

to:

AI agent.

Example:

```
Customer request
       ↓
Plan
       ↓
CRM
       ↓
Billing
       ↓
RAG
       ↓
Policy
       ↓
Workflow
       ↓
Response
```

Week 5 — RAG + Memory + Policies

Build:

```
Knowledge ingestion
Retrieval
Citations
Long-term customer memory
Policy engine
Guardrails
Approval rules
```

Now the agent can perform much more realistic enterprise interactions.

Week 6 — Command Center

Build:

```
Live conversation dashboard
Conversation timeline
Tool execution view
RAG source view
Memory view
Workflow view
Analytics
Call summary
```

This is when the architecture becomes visually demonstrable.

Week 7 — Robustness

Focus on:

```
Latency
Retries
Timeouts
Tool failures
LLM failures
Interruptions
Barge-in
Session recovery
Human escalation
```

Also add demo data and failure scenarios.

Week 8 — Demo hardening

Build 3–5 polished scenarios:

```
1. Technical support
2. Billing + refund
3. Cancellation / retention
4. Complex multi-intent complaint
5. Human escalation
```

Then prepare:

```
Demo scripts
Metrics
Architecture view
Command Center dashboards
Failure demonstrations
```

---

22. MVP versus final demo

Don't try to build every contact-center feature immediately.

MVP

The minimum credible system is:

```
WebRTC
+
STT
+
Agent Planner
+
```

```
Memory
+
3–5 Tools
+
RAG
+
Workflow Executor
+
TTS
```

With one complete workflow such as:

```
Technical Issue
    ↓
CRM lookup
    ↓
Outage check
    ↓
Diagnostics
    ↓
Engineer booking
    ↓
Ticket creation
```

Full demo

Add:

```
Multiple workflows
Multi-step planning
Long-term memory
Policy engine
Sentiment
Human escalation
Command Center
```

```
Call summaries
Analytics
Failure handling
```

---

23. What should NOT be built

For this demo, do not spend engineering time building:

```
Actual telecom infrastructure
Actual PSTN integration
Full CRM product
Full ticketing product
Full workforce management
Production-grade identity platform
Enterprise IAM platform
Huge vector database infrastructure
Complex Kubernetes platform
```

Mock the enterprise systems behind clean APIs.

The audience cares that:

```
AI → CRM → result
```

works reliably, not that you recreated Salesforce.

---

24. Definition of done

I would define the demo as successful when a user can say something like:

> "My internet stopped working yesterday, the bill also has an extra charge I don't understand, and I'm considering cancelling because this has happened before."

and the system can:

```
✓ Understand multiple intents
✓ Retrieve customer
✓ Remember prior issue
✓ Check service status
✓ Retrieve invoice
✓ Search relevant policy
✓ Dynamically plan next actions
✓ Call multiple enterprise tools
✓ Execute a deterministic workflow when required
✓ Maintain state across turns
✓ Detect sentiment
✓ Apply policy
✓ Explain results naturally
✓ Create ticket if necessary
✓ Schedule engineer
✓ Escalate when required
✓ Produce a call summary
✓ Show the complete execution timeline
```

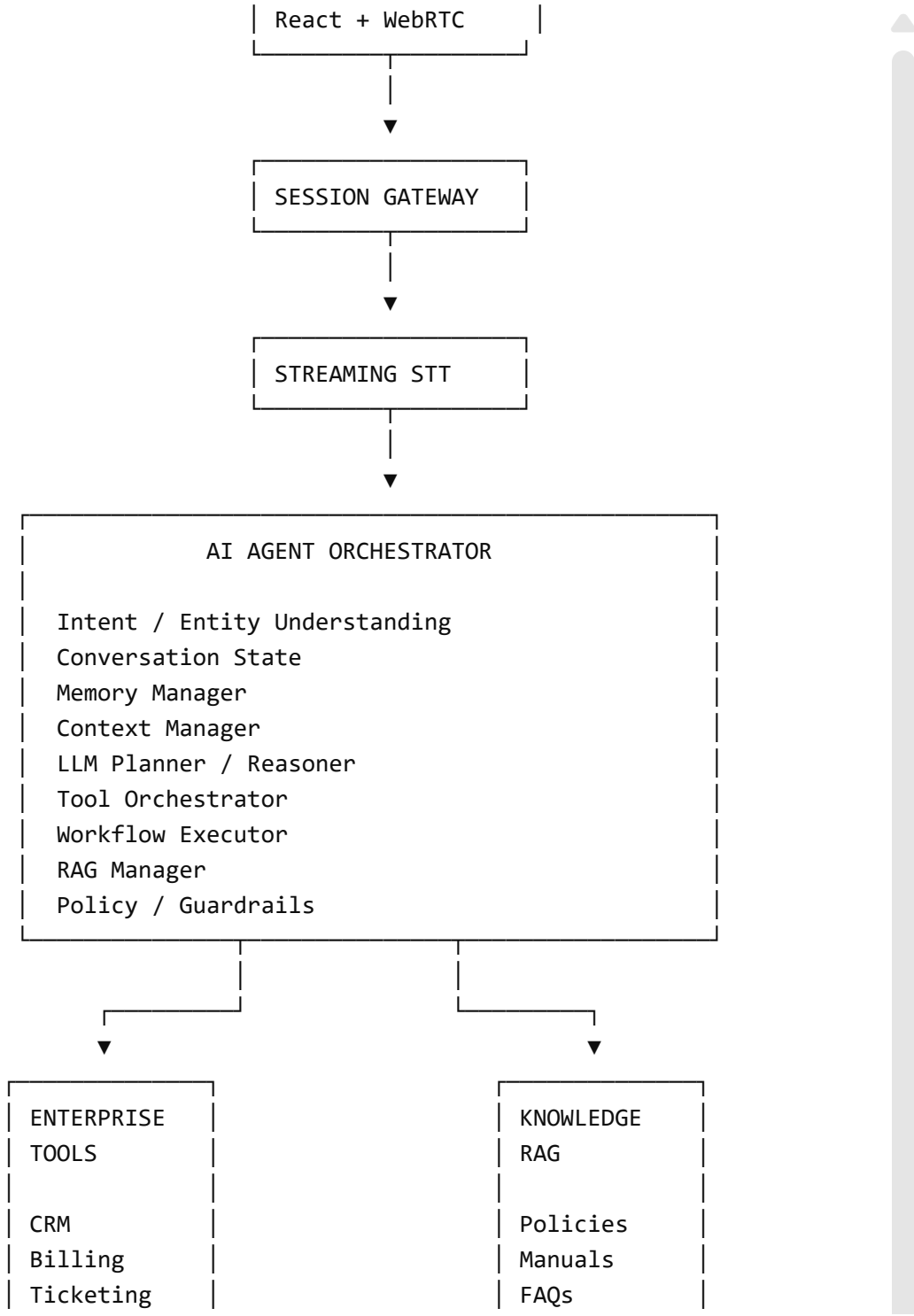
That single scenario demonstrates almost the entire architecture.

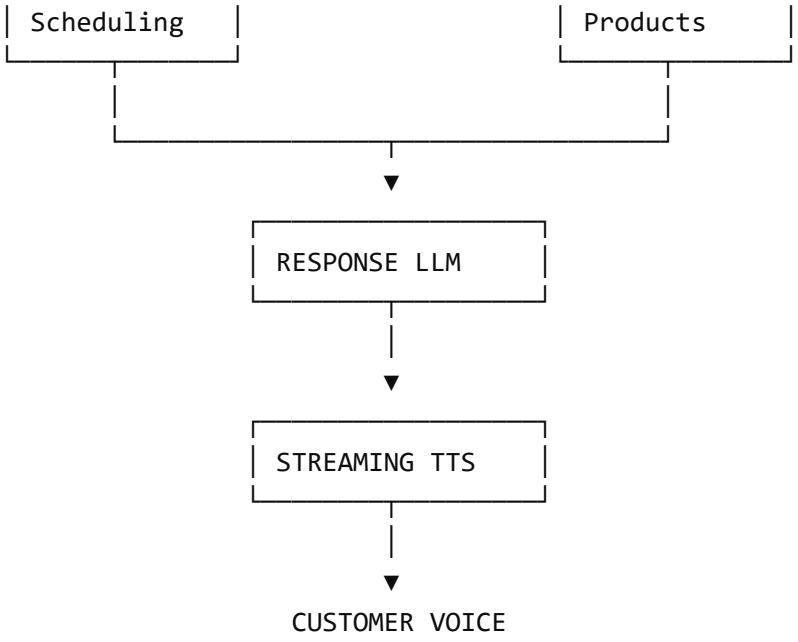
---

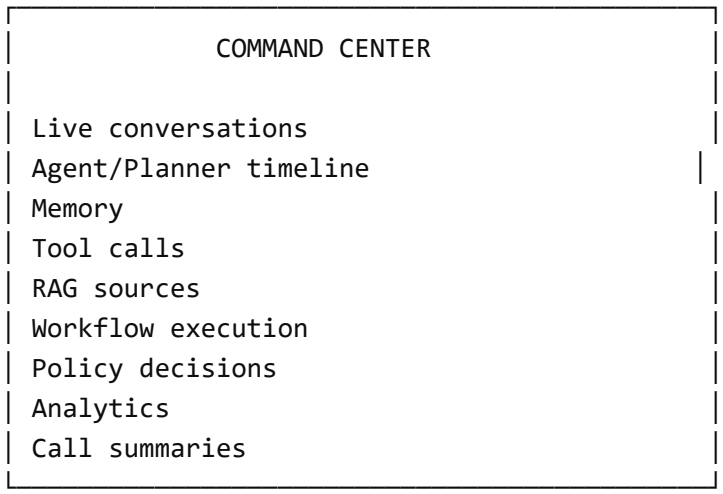
25. Final implementation view

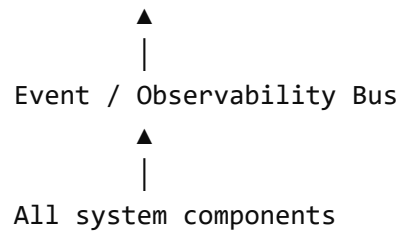
The architecture I would freeze for development is:

```
┌──────────────────┐
│  CUSTOMER BROWSER  │
```


React + WebRTC
SESSION GATEWAY
STREAMING STT
AI AGENT ORCHESTRATOR
Intent / Entity Understanding
Conversation State
Memory Manager
Context Manager
LLM Planner / Reasoner
Tool Orchestrator
Workflow Executor
RAG Manager
Policy / Guardrails
ENTERPRISE TOOLS
CRM
Billing
Ticketing
KNOWLEDGE RAG
Policies
Manuals
FAQs


Scheduling
Products
RESPONSE LLM
STREAMING TTS
CUSTOMER VOICE


COMMAND CENTER
Live conversations
Agent/Planner timeline
Memory
Tool calls
RAG sources
Workflow execution
Policy decisions
Analytics
Call summaries


Event / Observability Bus
All system components

The most important development decision is to **build the event model and orchestrator contracts first**. Once those are stable, STT, LLM, tools, RAG, workflows, and the Command Center can evolve independently. This also makes the timeline much safer because multiple developers can work in parallel rather than creating one large tightly coupled voice-bot codebase.